

แบบทดสอบภาคปฏิบัติ: ระบบ Spring Boot - Customer-Order-OrderLine-Product

ชื่อ: ดร.ณัฏฐา สว่างใส

รหัสนักศึกษา: 6633 80237-2

วันที่: 16 ตุลาคม 2568

คะแนนรวม: 120 คะแนน

ตอนที่ 1 Implementation (50 คะแนน)

ส่วนที่ 1: จงพัฒนา Restfull Api เพื่อการอ่าน ค้นหา บันทึก เพิ่ม ปรับปรุง ลบ ข้อมูลของโม order ซึ่งสัมพันธ์กับ Customer, OrderLine, Product ด้วย spring framework ส่ง เป็น ไฟล์ zip ของโปรเจ็ค (30 คะแนน)

ส่วนที่ 2 ให้แสดงการทดสอบ โดยใช้ PostMan ผ่านคลิกสั้น ๆ ให้ครบทุก url การค้นหาทั้งหมด ค้นหาโดยรหัส เพิ่มใบคำสั่งซื้อใหม่ ปรับปรุง ลบ (10 คะแนน)

ส่วนที่ 3 ให้อธิบายคำสั่งในส่วนต่าง ๆ ที่สำคัญโดยเฉพาะ Service และ Controller โดยให้เห็นหน้าผู้อธิบายด้วย บันทึกลงคลิก สั้น ๆ ไม่เกิน 5 นาที (10 คะแนน)

ตอนที่ 2 ตอบคำถามให้เขียนด้วยลายมือ และส่งเป็นไฟล์ pdf

ส่วนที่ 1: คำถามเชิงแนวคิด (20 คะแนน)

1. จงอธิบายความสัมพันธ์ระหว่างเอนทิตี Customer, Order, OrderLine และ Product ในระบบ Spring Boot

ตอบ: Customer → Order : One-to-Many ลูกค้า 1 คน มีได้ หลาย Order
Order → Customer : Many-to-One Order แต่ละอันจะอ้างอิง กับ 1 คน
Order → OrderLine : One-to-Many 1 Order มีหลาย สินค้า
OrderLine → Order : Many-to-One แต่ละ Order Line เป็นส่วนหนึ่งของ Order 1 รายการ
OrderLine → Product : Many-to-One แต่ละ รายการ (บรรทัด) ระบุสินค้า 1 ชิ้น

2. จงอธิบายความแตกต่างระหว่าง @OneToMany และ @ManyToOne ใน JPA คืออะไร? ยกตัวอย่างจากระบบของคุณ

ตอบ: 1 Entity เชื่อมกับ หลาย Entity เป็น @OneToMany(mappedBy = "order" ...)
private List<OrderLine> ol = new ArrayList<>();
1 order มีหลาย OrderLine
หลาย Entity เชื่อมกับ 1 Entity เป็น class Orderline {
@ManyToOne @JoinColumn(name = "order_id")
private Order order;

3. จงอธิบายว่าเหตุใดจึงใช้ @JoinColumn(name = "customer_id") ในคลาส Order และจะเกิดอะไรขึ้นหากไม่ใช้คำสั่งนี้

ตอบ: ระบุชื่อคอลัมน์ใน Database ถ้าไม่ใช้ JPA สืบชื่อ column 10 ทำให้ DB ล้ม Code หนัก
ตัวกัน ข้อ Error เมื่อมีข้อผิดพลาด

4. จงอธิบายว่าควรใช้ CascadeType.ALL ในความสัมพันธ์ของเอนทิตี และมีความเสี่ยงอย่างไร

คำตอบ: เช่นใน Order หากมีการลบ Order จะพ่วงลบ OrderLine ด้วยอัตโนมัติ ทำให้ลบ Order ไม่ทัน เรียง OrderLine อีก ความเสี่ยง: ถ้าลบ Order → OrderLine ถูกลบ

5. ทำไมคลาส Order จึงใช้ @Table(name = "orders")? และช่วยแก้ปัญหาอะไร

คำตอบ: กำหนดชื่อที่แน่นอน เพื่อไม่ให้ JPA สับสน เราอัตโนมัติ และ เป็น keyword ของ Database

ส่วนที่ 2: ความเข้าใจเกี่ยวกับ JSON และ API (20 คะแนน)

6. จงเขียน JSON เพื่อสร้าง Order ใหม่ที่เชื่อมกับ Customer (ID = 3) และมีสินค้า 2 รายการ (ID = 1 และ 2)

โดยแต่ละสินค้าระบุจำนวนและราคา

คำตอบ: {
 "order Date": "2025-10-18",
 "customer": { "id": 3 },
 "orderLines": {
 { "quantity": 2, "product": { "id": 3, "price": 50.00 },
 { "quantity": 1, "product": { "id": 2, "price": 30.00 },

7. เมื่อส่ง JSON ข้างต้น Spring Boot จะทำการแมปข้อมูล product และ customer แบบซ้อน (nested) ไปยังอ็อบเจกต์ Entity ได้อย่างไร

คำตอบ: ถ้า JSON มี Id Spring จะหา entity จาก Database / ถ้าไม่มี Spring จะสร้าง entity ใหม่ และเซฟลง Database

8. หาก API ตอบกลับด้วย “400 Bad Request” ระบุสาเหตุที่อาจเกิดขึ้นได้อย่างน้อย 3 ข้อ

คำตอบ: ฟิลด์ ที่จำเป็นขาดหายไป, ชนิดข้อมูลไม่ตรง, โครงสร้าง JSON ผิด

9. จง แสดงตัวอย่าง JSON ที่ได้อากการเรียกใช้ GET /api/orders/1 และอธิบายส่วนประกอบของผลลัพธ์แต่ละส่วน

คำตอบ:

```
"id": 1,
"orderDate": "2025-10-18",
"customer": {
  "id": 3,
  "name": "Bob",
  "email": "bob@gmail.com"
},
"orderLines": [
  {
    "id": 1,
    "quantity": 2,
    "product": { "id": 1, "name": "Coke", "price": 50.00 }
  },
  {
    "id": 2,
    "quantity": 1,
    "product": { "id": 2, "name": "Pepsi", "price": 30.00 }
  }
]
```

ส่วนที่ 3: การเขียนโค้ดและการแก้ไขปัญหา (15 คะแนน)

10. พิจารณาโค้ดต่อไปนี้ในคลาส OrderController:

```
@PostMapping
public Order createOrder(@RequestBody Order order) {
    return orderRepository.save(order);
}
```

จงอธิบายเหตุผลว่าโค้ดนี้อาจล้มเหลวหาก customer หรือ product ใน order ยังไม่ถูกบันทึกในฐานข้อมูล

คำตอบ: การเรียก orderRepository.save(order);

จะเกิด foreign key constraint violation เพราะฐานข้อมูลจำเป็นต้องมี customer_id และ product_id อยู่แล้ว

11. ในคลาส OrderLine จะเกิดอะไรขึ้นหากมีใช้คำสั่ง @ManyToOne และ @JoinColumn(name = "order_id")
เมื่อประกาศตัวแปร Order

คำตอบ: JPA จะไม่รู้จัก จนกระทั่งเห็นชื่อของ Order กับ OrderLine

12. คำสั่งในไฟล์ application.properties ได้ใช้ควบคุมการสร้างตารางอัตโนมัติ
และเขียนบรรทัดคำสั่งที่ถูกต้องเพื่อเปิดใช้งานการอัปเดต schema อัตโนมัติใน MySQL

คำตอบ: spring.jpa.hibernate.ddl-auto= update

ส่วนที่ 4: การออกแบบและประยุกต์ใช้งาน (20 คะแนน)

13. กรณีต้องการค้นหาโดยรหัสของลูกค้า จงเขียนฟังก์ชันของ RestController เพื่อแสดงผลลัพธ์ของใบคำสั่งซื้อทั้งหมดของลูกค้ารายนั้น
พร้อมระบุ url ที่ผู้ใช้งานต้องป้อนเพื่อให้ผลลัพธ์ดังกล่าว

คำตอบ: @GetMapping("/customer/{customerId}")

```
public List<Order> getOrderByCustomer(@PathVariable long customerId){  
    return orderService.findByCustomerId(customerId);  
}
```

url : http://.../api/orders/customer/{customerId}

14. สมมติว่าคุณต้องการคำนวณมูลค่ารวมของคำสั่งซื้อ (รวมราคาของทุกบรรทัด) เขียนโค้ด Java สำหรับเมธอดใน Service เพื่อคำนวณจากข้อมูลในเอนทิตี Order

คำตอบ: public double calculateTotal(Order order) {
 return order.getOrderLines().stream().mapToDouble(line → line.getQuantity()
 * line.getProduct().getPrice()).sum();
}

